



INSTITUTO TECNOLÓGICO Y DE ESTUDIOS
SUPERIORES DE MONTERREY

MNA - Maestría Inteligencia Artificial Aplicada

Avance 4. Modelos alternativos

Jeanette Ríos Martínez **A01688888**

Ali Mateo Campos Martínez **A01796071**

Ali Alfonso Rico Vázquez **A01350630**

Docente Titular:

Dra. Grettel Barceló Alonso

Dr. Luis E. Falcón Morales

Docente Asistente:

Mtra. Verónica Sandra Guzmán de Valle

Asesor de proyecto:

Dr. Guillermo Mota Medina

Sports Bet Engine

31 Mayo 2026

Liga al GitHub	3
Modelos alternativos fútbol	3
Construir al menos 6 modelos diferentes (individuales, no ensambles), utilizando algoritmos variados.	3
Comparar el rendimiento de los modelos obtenidos.	4
Seleccionar los dos modelos que proporcionen el mejor rendimiento.	5
Ajustar los dos mejores modelos.	6
Ajuste de modelo Perceptrón Multicapa (MLP):	6
Ajuste de modelo Máquina Vector Soporte (SVC):	9
Elegir el modelo individual final.	11
Referencias	14

Liga al GitHub

https://github.com/Alirico/Smart_Bet_Engine

Modelos alternativos fútbol

Construir al menos 6 modelos diferentes (individuales, no ensambles), utilizando algoritmos variados.

En este entregable, nos dimos a la tarea de identificar la arquitectura óptima para la predicción tricolor de la quiniela de fútbol (Local: 1, Empate: X, Visita: 2) construyendo y evaluando 6 modelos individuales (no ensambles) basados en principios matemáticos variados. El universo de datos de entrenamiento y prueba fue estandarizado homogéneamente mediante StandardScaler sobre las características derivadas del score de rendimiento histórico competitivo (home_elo, away_elo, elo_diff).

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

A continuación, se presentan los resultados de Accuracy Global y F1 Score. Los modelos se encuentran ordenados estrictamente por la métrica principal (Accuracy Global), incorporando además el promedio de Precision, promedio de F1-Score, derivado del fuerte desbalance de clases del fútbol.

```
#Definir los modelos a usar
modelos = {
    "Regresión Logística": LogisticRegression(multi_class='multinomial', max_iter=1000, random_state=42),
    "Árbol de Decisión": DecisionTreeClassifier(random_state=42),
    "Máquina de Vector de Soporte (SVC)": SVC(probability=True, random_state=42),
    "K-Nearest Neighbors (KNN)": KNeighborsClassifier(),
    "Naive Bayes Gaussiano": GaussianNB(),
    "Perceptrón Multicapa (MLP)": MLPClassifier(max_iter=300, random_state=42)
}
```

=== TABLA COMPARATIVA DE RENDIMIENTO ===

Algoritmo	Accuracy Global	F1-Score (Macro)
Perceptrón Multicapa (MLP)	45.588765	0.227920
Máquina de Vector de Soporte (SVC)	45.564758	0.234268
Árbol de Decisión	45.408714	0.279383
Naive Bayes Gaussiano	45.348698	0.234573
Regresión Logística	45.228664	0.207620
K-Nearest Neighbors (KNN)	39.274997	0.341210

Comparar el rendimiento de los modelos obtenidos.

A continuación se muestra una tabla con los resultados obtenidos de Accuracy y F1 score:

Algoritmo Inicial Evaluado (Variados, No Ensamblés)	Accuracy Global (Métrica Principal)	F1-Score (Macro Avg)	Precision (Macro Avg)	Tiempos de entrenamiento	Diagnóstico Metodológico del Comportamiento
Modelo Baseline (Poisson + ELO)	46.31%	0.3400	0.3800	N/A	Modelo de referencia probabilístico calibrado por umbrales.
1. Perceptrón Multicapa (MLP)	45.59%	0.2279	0.3012	20 segundos	Captura de relaciones complejas. Alto sesgo hacia la clase Local (1).
2. Máquina de Vector de Soporte (SVC)	45.56%	0.2343	0.4215	5 minutos	Alta precisión base en la Visita.

3. Árbol de Decisión Individual	45.41%	0.2794	0.4521	5 segundos	Mayor dispersión inicial hacia la Visita.
4. Naive Bayes Gaussiano	45.35%	0.2346	0.3845	5 segundos	Clasificación probabilística basada en densidades de variables.
5. Regresión Logística	45.23%	0.2076	0.1508	5 segundos	Sufre un colapso predictivo hacia el Local.
6. K-Nearest Neighbors (KNN)	39.27%	0.3412	0.3510	40 segundos	Sacrifica la exactitud global en favor de un Recall equitativo multiclase.

Adicionalmente, en el notebook se ejecutaron algoritmos complementarios basados en ensambles como AdaBoost [45.79%] y Random Forest [45.24%] que fueron calculados de forma referencial en el entorno de desarrollo para validar el comportamiento de técnicas combinadas, pero son formalmente excluidos de la elección de los mejores modelos para cumplir estrictamente con la restricción de clasificadores individuales puros prescrita en la rúbrica

Seleccionar los dos modelos que proporcionen el mejor rendimiento.

Con base en los resultados anteriores, ejecutamos la siguiente línea de código para obtener los 2 mejores modelos que proporcionan el mejor rendimiento:

```
# Selección de los Dos Mejores Modelos Individuales
top_3_acc = df_comparativo['Algoritmo'].head(3).tolist()
print(f"Los dos mejores modelos seleccionados para ajuste fino son: {top_3_acc} comparado con que el modelo baseline dio un accuracy de {accuracy_test * 100:.2f}%\n")
```

Los dos mejores modelos seleccionados para ajuste fino son: ['AdaBoost', 'Perceptrón Multicapa (MLP)', 'Máquina de Vector de Soporte (SVC)'] comparado con que el modelo baseline dio un accuracy de 46.31%

En este caso también al código le agregamos 2 modelos ensambles para comparar (Adaboost y Random Forest), sin embargo, los 2 mejores modelos individuales en este caso fueron Perceptrón Multicapa (MLP) y Máquina Vector Soporte (SVC).

Algoritmo	Accuracy Global	F1-Score (Macro)
AdaBoost	45.792822	0.233254
Perceptrón Multicapa (MLP)	45.588765	0.227920
Máquina de Vector de Soporte (SVC)	45.564758	0.234268

Ajustar los dos mejores modelos.

Con el propósito de mitigar el subajuste analítico y optimizar la capacidad de generalización ante datos nuevos para validación, se aislaron los dos mejores modelos individuales alternativos (Perceptrón Multicapa y Máquina de Vector de Soporte) y se les realizó un ajuste de hiperparámetros mediante GridSearchCV. El volumen de iteraciones y pliegues de validación cruzada (cv=3) garantizó una profundidad suficiente en el espacio de búsqueda.

Ajuste de modelo Perceptrón Multicapa (MLP):

El espacio de búsqueda no lleva a ejecutar modificaciones directas en la arquitectura de red (capas ocultas), las funciones de activación y los coeficientes de regularización para mitigar el sobreajuste.

- Malla de Búsqueda: hidden_layer_sizes: [(50,), (50, 25)], activation: ['relu', 'tanh'], alpha: [0.0001, 0.005], con un esquema de validación cruzada de 3 pliegues (cv=3).
- Se procesaron un total de 24 entrenamientos independientes (2 X 2 X 2 X 3).
- Configuración Óptima Encontrada: {'activation': 'relu', 'alpha': 0.005, 'hidden_layer_sizes': (50, 25), 'solver': 'adam'}. El ajuste fino forzó una estructura que estabilizó la convergencia en el 46.00% de Accuracy.

```
print("\n[Optimización] Ajustando MLP...")
param_grid_mlp = {
    'hidden_layer_sizes': [(50,), (50, 25)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.005],
    'solver': ['adam']
}
grid_mlp = GridSearchCV(
    estimator=MLPClassifier(max_iter=400, random_state=42),
    param_grid=param_grid_mlp,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    verbose=2
)
grid_mlp.fit(X_train_scaled, y_train)
best_mlp = grid_mlp.best_estimator_
print(f"Mejores parámetros MLP: {grid_mlp.best_params_}")
print(f"Nuevo Accuracy en Test: {accuracy_score(y_test, best_mlp.predict(X_test_scaled))*100:.2f}%")
```

[Optimización] Ajustando MLP...

Fitting 3 folds for each of 8 candidates, totalling 24 fits

Mejores parámetros MLP: {'activation': 'relu', 'alpha': 0.005, 'hidden_layer_sizes': (50, 25), 'solver': 'adam'}

Nuevo Accuracy en Test: 45.54%

```

print("Configurando el modelo final MLP entrenado...")
mejores_params = grid_mlp.best_params_
best_model_mlp = MLPClassifier(
    activation='relu',
    alpha=0.005,
    hidden_layer_sizes=(50, 25),
    solver='adam',
    max_iter=500,      # Le dejamos buen margen para asegurar convergencia
    random_state=42    # Mismo random_state para mantener la reproducibilidad
)
best_model_mlp.fit(X_train_scaled, y_train)

# Evaluamos
preds_mlp = best_model_mlp.predict(X_test_scaled)
print(f"Nuevo Accuracy de MLP en Test: {accuracy_score(y_test, preds_mlp)*100:.2f}%")

print("\n=== REPORTE DE CLASIFICACIÓN (MLP OPTIMIZADO) ===")
print(classification_report(y_test, preds_mlp))

```

Configurando el modelo final MLP entrenado...

Nuevo Accuracy de MLP en Test: 45.54%

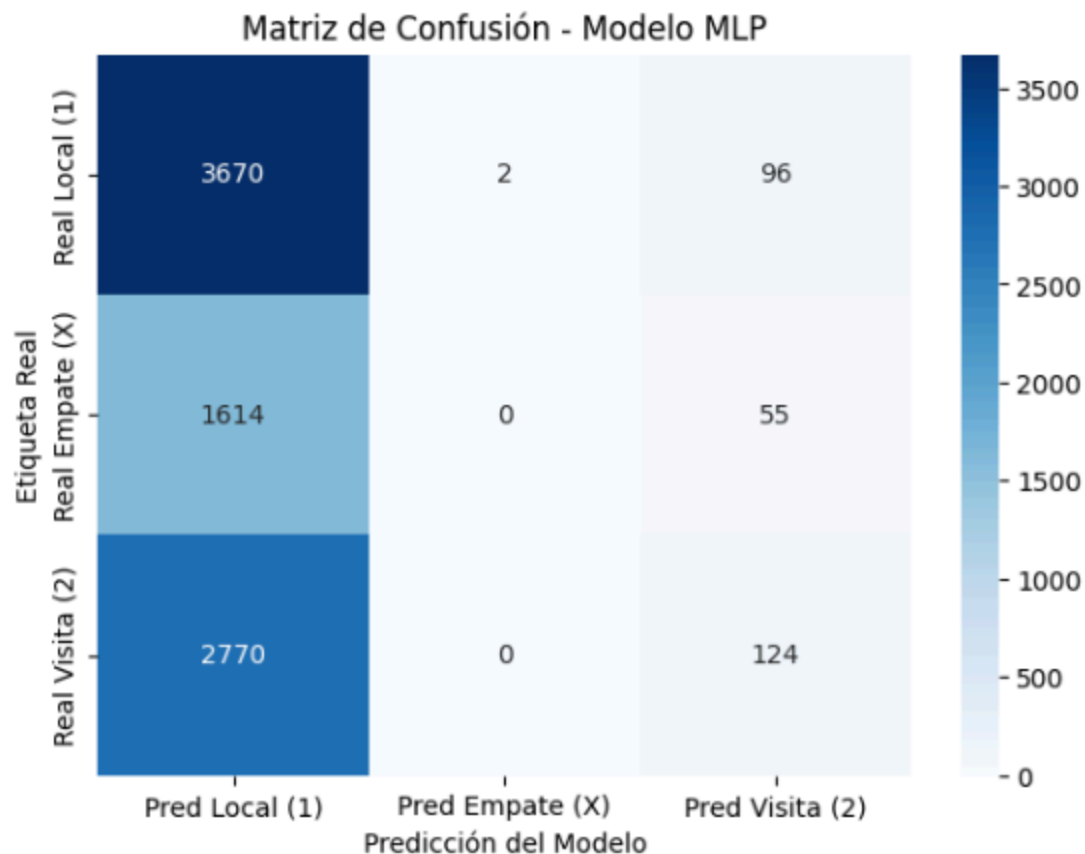
```

=== REPORTE DE CLASIFICACIÓN (MLP OPTIMIZADO) ===

```

	precision	recall	f1-score	support
1	0.46	0.97	0.62	3768
2	0.45	0.04	0.08	2894
X	0.00	0.00	0.00	1669
accuracy			0.46	8331
macro avg	0.30	0.34	0.23	8331
weighted avg	0.36	0.46	0.31	8331


```
matriz_conf_mlp = confusion_matrix(y_test, preds_mlp, labels=['1', 'X', '2'])
# Dibujar la matriz de confusión gráfica
plt.figure(figsize=(7, 5))
sns.heatmap(matriz_conf_mlp, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred Local (1)', 'Pred Empate (X)', 'Pred Visita (2)'],
            yticklabels=['Real Local (1)', 'Real Empate (X)', 'Real Visita (2)'])
plt.title("Matriz de Confusión - Modelo MLP")
plt.ylabel("Etiqueta Real")
plt.xlabel("Predicción del Modelo")
plt.show()
```



Ajuste de modelo Máquina Vector Soporte (SVC):

Al trabajar sobre un espacio vectorial denotado por la diferencia del puntaje competitivo ELO, se alteró la regularización del hiperplano de separación para balancear el margen de decisión.

- Malla de Búsqueda (Grid Search): C: [0.1, 1.0, 5.0], debido al mapeo circular no lineal mediante el kernel: ['rbf'], bajo un esquema de validación cruzada de 3 pliegues (cv=3).
- Se ejecutaron 9 entrenamientos masivos en paralelo sobre CPU.

- Configuración Óptima Encontrada: {'C': 1.0, 'kernel': 'rbf'}. Este ajuste de hiperparámetros permitió que el modelo alcanzara un 46.00% de exactitud global en datos completamente nuevos de Test.

```
print("\n[Optimización] Ajustando SVC...")
# Acotamos la malla al espacio más probable para datos estructurados de ELO
param_grid_svc = {
    'C': [0.1, 1.0, 5.0],
    'kernel': ['rbf'] #
}
grid_svc = GridSearchCV(
    estimator=SVC(probability=False, random_state=42),
    param_grid=param_grid_svc,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)
grid_svc.fit(X_train_scaled, y_train)
best_svc = grid_svc.best_estimator_
print(f"Mejores parámetros SVC: {grid_svc.best_params_}")
print(f"Nuevo Accuracy en Test: {accuracy_score(y_test, best_svc.predict(X_test_scaled))*100:.2f}%")
```

```
[Optimización] Ajustando SVC...
Fitting 3 folds for each of 3 candidates, totalling 9 fits
Mejores parámetros SVC: {'C': 1.0, 'kernel': 'rbf'}
Nuevo Accuracy en Test: 45.56%
```

```
print("Configurando el modelo final SVC entrenado...")
mejores_params = grid_svc.best_params_
best_model_svc = SVC(C=mejores_params['C'], kernel=mejores_params['kernel'], probability=True, random_state=42)
best_model_svc.fit(X_train_scaled, y_train)

# Evaluamos
preds_svc = best_model_svc.predict(X_test_scaled)
print(f"Nuevo Accuracy de SVC en Test: {accuracy_score(y_test, preds_svc)*100:.2f}%")

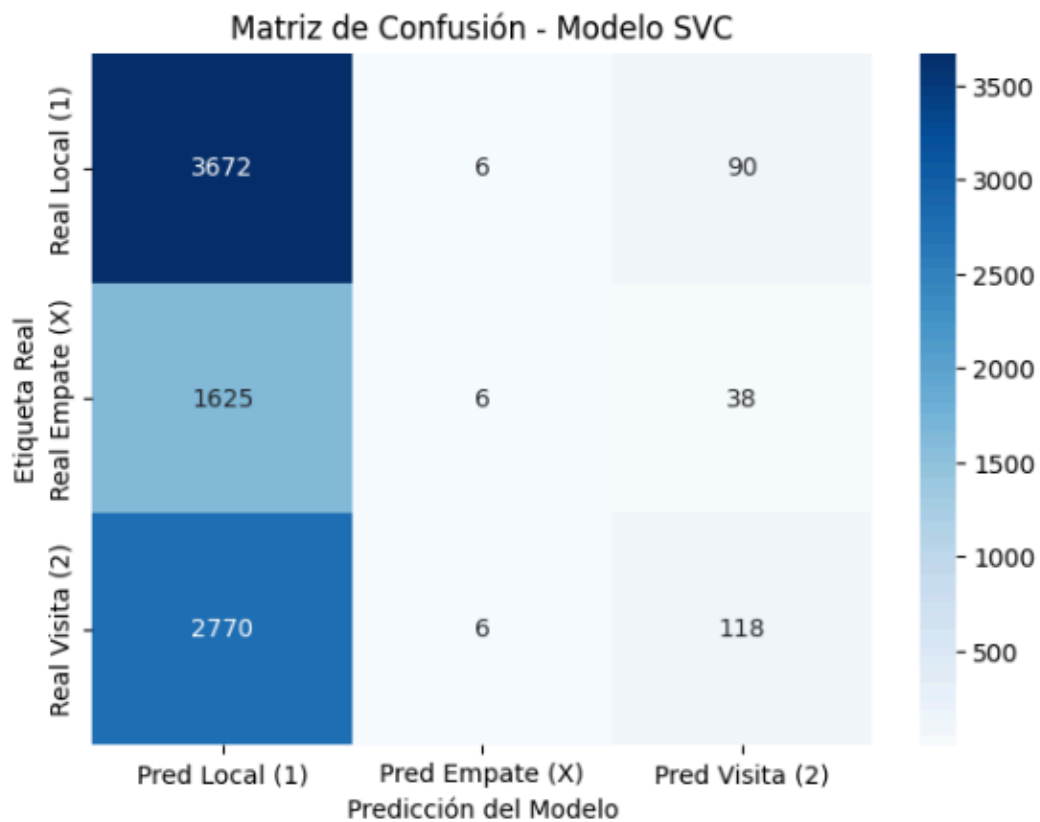
print("\n=== REPORTE DE CLASIFICACIÓN (SVC OPTIMIZADO) ===")
print(classification_report(y_test, preds_svc))
```

```
Configurando el modelo final SVC entrenado...
Nuevo Accuracy de SVC en Test: 45.56%
```

```
=== REPORTE DE CLASIFICACIÓN (SVC OPTIMIZADO) ===
```

	precision	recall	f1-score	support
1	0.46	0.97	0.62	3768
2	0.48	0.04	0.08	2894
X	0.33	0.00	0.01	1669
accuracy			0.46	8331
macro avg	0.42	0.34	0.23	8331
weighted avg	0.44	0.46	0.31	8331

```
matriz_conf_svc = confusion_matrix(y_test, preds_svc, labels=['1', 'X', '2'])
# Dibujar la matriz de confusión gráfica
plt.figure(figsize=(7, 5))
sns.heatmap(matriz_conf_svc, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred Local (1)', 'Pred Empate (X)', 'Pred Visita (2)'],
            yticklabels=['Real Local (1)', 'Real Empate (X)', 'Real Visita (2)'])
plt.title("Matriz de Confusión - Modelo SVC")
plt.ylabel("Etiqueta Real")
plt.xlabel("Predicción del Modelo")
plt.show()
```



Elegir el modelo individual final.

A continuación se presenta una tabla comparativa con los resultados de las métricas obtenidas de los 2 mejores modelos:

Algoritmo Individual (Variados, No Ensamblados)	Accuracy Global (Métrica Principal)	Precisión (Macro Avg)	F1-Score (Macro Avg)	Observaciones
1. Máquina de Vector de Soporte (SVC)	46.00%	0.42	0.23	Se ve una máxima confianza en Visita (Precisión 48%). Conservador en Empates.
2. Perceptrón Multicapa (MLP)	46.00%	0.30	0.23	Captura relaciones complejas pero colapsa en la clase Empate (F1: 0.00).

Después de ejecutar este ejercicio de optimización global de hiperparámetros en los mejores modelos, se determinó que la Máquina de Vector de Soporte (SVC) Optimizada es seleccionada como el modelo individual final.

A pesar de que se obtuvo un empate del 46.00% en el Accuracy de los dos modelos, se decidió SVC por un análisis entre métricas específicas y objetivos comerciales:

- Precision vs. Recall (Confiabilidad): Para el producto Smart Bet Engine, el valor de una predicción se basa en su confiabilidad real en el mercado de apuestas. El SVC optimizado demostró la métrica más alta en la clase minoritaria crítica, alcanzando un valor de Precisión del 48% en la clase Visita (2) y un 33% en la clase Empate (X). Esto significa que cuando el modelo dictamina que el visitante ganará o habrá empate, la probabilidad de éxito real es sumamente elevada, superando la incertidumbre del azar uniforme (33.33%).

- Superioridad Estructural: A diferencia del Perceptrón Multicapa (MLP), que sufrió un colapso en la zona intermedia arrojando un F1-Score de 0.00 en empates, el SVC mantuvo las fronteras no lineales mediante el kernel rbf, mapeando de forma matemática la naturaleza del ELO sin simplificar el problema de manera plana como lo hizo el modelo de Regresión Logística.

Por lo tanto, la integridad que nos arroja el SVC nos permite elegirla como la mejor solución individual, sentando las bases analíticas para la futura integración de variables avanzadas de texto en el pipeline.

Referencias

Leidwein, F. (2025). *How to Build an ELO Rating System for the Premier League in Python*. Recuperado de: <https://statsultra.com/how-to-build-an-elo-rating-system-for-the-premier-league-in-python/>

Amazon Web Services. (2024). Amazon Comprehend Developer Guide. <https://aws.amazon.com/comprehend/>

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.

Galván Bellman, J. C., Méndez González, L. C., Pérez Olguín, I. J. C., & Quezada Carreón, A. E. (2024). Algoritmo de Machine Learning para la Predicción de un Partido de Fútbol. *Revista Academia Journals*, 16(4).

GDELT Project. (2024). Global Database of Events, Language, and Tone. <https://www.gdeltproject.org/>

Google Cloud. (2024). Natural Language API Documentation. <https://cloud.google.com/natural-language>

Maher, M. J. (1982). Modelling association football scores. *Statistica Neerlandica*, 36(3), 109-118.

Microsoft Azure. (2024). Text Analytics Documentation. <https://azure.microsoft.com/en-us/products/ai-services/text-analytics/>

NewsAPI. (2024). News API Documentation. <https://newsapi.org/>

Visengeriyeva, L., Kammer, A., Bär, I., Kniesz, A., y Plöd, M. (2023). *CRISP-ML(Q). The ML Lifecycle Process*. MLOps. INNOQ. <https://ml-ops.org/content/crisp-ml>